CUEE203: OBJECT-ORIENTED PROGRAMMING

PRACTICAL RECORD

Enrollment No.	25CE015
Name	Dhairy Chauhan

Practical 7: Form Validator using Annotations and Reflection

Description

1. Define @NotBlank and @MaxLength annotations, apply them to SignupForm fields, and use reflection to report validation errors.

Code

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;
import java.util.Scanner;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

class UserForm {
    @NotBlank
    String name;

    @MaxLength(10)
    String city;

    UserForm(String name, String city) {
        this.name = name;
        this.city = city;
    }
}

public class pract7 {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your city: ");
        String city = sc.nextLine();

        UserForm user = new UserForm(name, city);
        List<String> errors = new LinkedList<>();

        for (Field field : user.getClass().getDeclaredFields()) {
            field.setAccessible(true);
            Object value = field.get(user);

            if (field.isAnnotationPresent(NotBlank.class)) {
                if (value == null || value.toString().trim().isEmpty()) {
                    errors.add(field.getName() + " cannot be blank");
```

```
                }
            }

            if (field.isAnnotationPresent(MaxLength.class)) {
                int max = field.getAnnotation(MaxLength.class).value();

                if (value != null && value.toString().length() > max) {
                    errors.add(field.getName() + " must not exceed " + max + "
characters");
                }
            }
        }

        if (errors.isEmpty()) {
            System.out.println("Form is valid");
        } else {
            System.out.println("Form validation failed:");
            for (String error : errors) {
                System.out.println(error);
            }
        }
        System.out.println("Dhairy Chauhan 25CE015");

        sc.close();
    }
}
```

Output

```
PS D:\JAVA_LAB-main\P7\Form Validator> javac main.java
PS D:\JAVA_LAB-main\P7\Form Validator> java main
Enter your name: Dhairy
Enter your city: morbi
Form is valid
Dhairy Chauhan 25CE015
PS D:\JAVA_LAB-main\P7\Form Validator>
```

P-1.1.png

Program 7.2: Mini Test Runner

Description	2. Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods
Code	

```
import java.lang.annotation.*;

import java.lang.reflect.*;



@Retention(RetentionPolicy.RUNTIME)

@Target(ElementType.METHOD)
```

```
@interface Run {}



class TestCases {

    @Run

    public void testAddition() {

        System.out.println("Addition test passed");

    }



    @Run

    public void testSubtraction() {

        System.out.println("Subtraction test passed");

    }



    public void ignoredMethod() {

        System.out.println("This method should not run");

    }

}



public class main {

    public static void main(String[] args) throws Exception {

        TestCases tests = new TestCases();

        int count = 0;



        for (Method method : tests.getClass().getDeclaredMethods()) {

            if (method.isAnnotationPresent(Run.class)) {

                method.invoke(tests);

                count++;

            }

        }
```

```
        System.out.println("Tests run: " + count);

        System.out.println("Dhairy Chauhan 25CE015");

    }

}
```

Output

```
PS D:\JAVA_LAB-main\P7\Mini Test Runner> javac main.java
PS D:\JAVA_LAB-main\P7\Mini Test Runner> java main
Addition test passed
Subtraction test passed
Tests run: 2
Dhairy Chauhan 25CE015
PS D:\JAVA_LAB-main\P7\Mini Test Runner>
```

p-1.2.png

Program 7.3: Object Mapping Using Reflection

Description

3. Define a `@Column(name)` annotation and use reflection to populate an object's fields by matching column names from a header row and a data row. Handle missing columns appropriately.

Code

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

class Student {
    @Column(name = "name")
    String name;

    @Column(name = "age")
    String age;

    @Column(name = "city")
    String city;
}

public class main{
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter column names separated by commas:");
        String[] header = sc.nextLine().split(",");

        System.out.println("Enter values in the same order:");
        String[] data = sc.nextLine().split(",", -1);

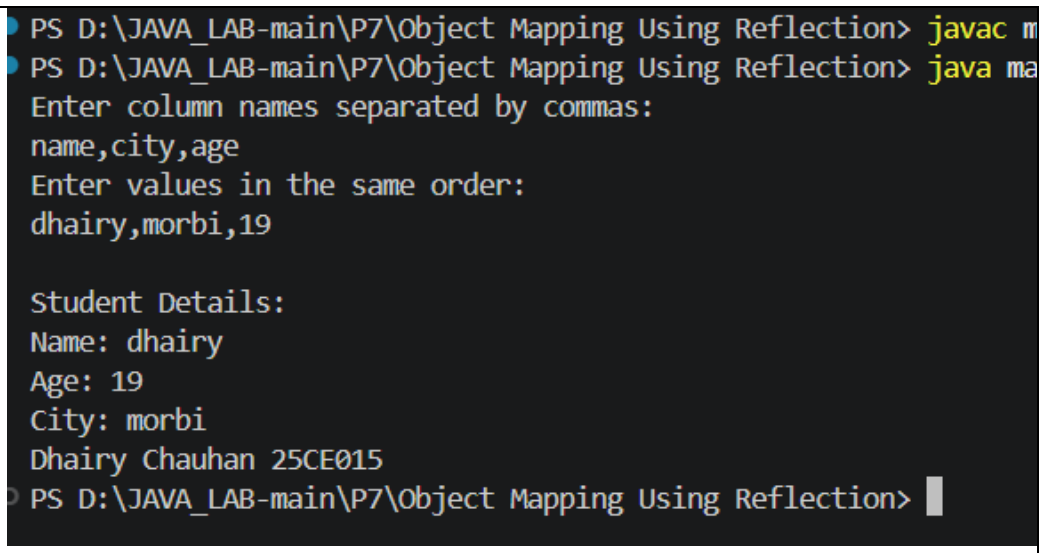
        if (header.length != data.length) {
            System.out.println("Error: Number of columns and values do not
match");
            sc.close();
            return;
        }
```

```
        Student s = new Student();

        for (Field f : Student.class.getDeclaredFields()) {
            Column c = f.getAnnotation(Column.class);

            if (c != null) {
                for (int i = 0; i < header.length; i++) {
                    if (header[i].trim().equals(c.name())) {
                        f.setAccessible(true);
                        f.set(s, data[i].trim());
                        break;
                    }
                }
            }
        }

        System.out.println("\nStudent Details:");
        System.out.println("Name: " +
            (s.name == null ? "Missing column" : s.name));
        System.out.println("Age: " +
            (s.age == null ? "Missing column" : s.age));
        System.out.println("City: " +
            (s.city == null ? "Missing column" : s.city));
         System.out.println("Dhairy Chauhan 25CE015");
        sc.close();
    }
}
```

Output	

Practical 8: Exception Handling

Description	1. To develop a calculator that handles invalid numeric input, unsupported operators, and division by zero using `try`, `catch`, and `finally`. The program repeats until a valid calculation is completed.

Code

```
import java.util.Scanner;
import java.util.InputMismatchException;

class DivideByZeroException extends Exception {
    public DivideByZeroException(String message) {
        super(message);
    }
}

public class main {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        boolean successful = false;

        while (!successful) {
            try {
                System.out.print("Enter first number: ");
                double num1 = sc.nextDouble();
                System.out.print("Enter operator: ");
                char op = sc.next().charAt(0);
                System.out.print("Enter second number: ");
                double num2 = sc.nextDouble();
                if (op == '/' && num2 == 0) {
                    throw new DivideByZeroException("Division by zero is mathe-
matically undefined.");
                }
                double result = 0;
                boolean valid = true;
                if (op == '+') result = num1 + num2;
                else if (op == '-') result = num1 - num2;
                else if (op == '*') result = num1 * num2;
                else if (op == '/') result = num1 / num2;
                else {
                    System.out.println("Error: Invalid operator chosen.");
                    valid = false;
                }
                if (valid) {
                    System.out.printf("Result: %.2f %c %.2f = %.2f\n", num1, op,
num2, result);
                    successful = true;
                }
            } catch (DivideByZeroException e) {
                System.out.println("Error: " + e.getMessage());
            } catch (InputMismatchException e) {
                System.out.println("Error: Invalid number input.");
                sc.nextLine();
            }
        }
        System.out.println("Dhairy Chauhan 25CE015");
        sc.close();
    }
}
```

Output	

```
PS D:\JAVA_LAB-main\P8\Guarded calculator> javac main.java
PS D:\JAVA_LAB-main\P8\Guarded calculator> java main
Enter first number: 15
Enter operator: +
Enter second number: 39
Result: 15.00 + 39.00 = 54.00
Dhairy Chauhan 25CE015
PS D:\JAVA_LAB-main\P8\Guarded calculator>
```

p-8.1.png

Program 8.2: Stock issue

Description	2. Use custom checked exceptions for insufficient stock and invalid quantities. Catch each failure so the remaining requests continue.

Code

```
import java.util.Scanner;

class Warehouse {
    public String item;
    public int stock;

    public Warehouse(String item, int stock) {
        this.item = item;
        this.stock = stock;
    }

    public void issue(String item, int qty) throws InvalidQuantityException, Ou-
tOfStockException {
        if (qty <= 0) {
            throw new InvalidQuantityException("Quantity must be greater than
zero. Requested: " + qty);
        }
        if (qty > stock) {
            throw new OutOfStockException("not enoght qty", qty - stock);
        }
        this.stock = this.stock - qty;
    }
}

class OutOfStockException extends Exception {
    public int shortfall;

    public OutOfStockException(String message, int shortfall) {
        super(message);
        this.shortfall = shortfall;
    }

    public int getShortfall() {
        return shortfall;
    }
}

class InvalidQuantityException extends Exception {
    public InvalidQuantityException(String message) {
        super(message);
    }
}

record Request(String item, int qty) {}

public class main {
    public static void main(String args[]) {
        Warehouse w = new Warehouse("Gada electronics", 2500000);
        Request[] requests = {
            new Request("TV", 5),
            new Request("mobile", 0),
            new Request("", 15),
            new Request("XBOX", 26),
            new Request("CPU", 3),
            new Request("FlipperZero", 2)
        };

        for (Request req : requests) {
            try {
                w.issue(req.item(), req.qty());
            } catch (InvalidQuantityException e) {
                System.err.println("Failure: " + e.getMessage());
            } catch (OutOfStockException e) {
                System.err.println("Failure: " + e.getMessage() + " Shortfall
amount: " + e.getShortfall());
                 System.out.println("Dhairy Chauhan 25CE015");
            }
        }
    }
```

```
}
```

Output

```
PS D:\JAVA_LAB-main\P8\Stock issue> javac main.java
PS D:\JAVA_LAB-main\P8\Stock issue> java main
Failure: Quantity must be greater than zero. Requested: 0
PS D:\JAVA_LAB-main\P8\Stock issue>
```

p-8.2.png

Program 8.3: Advance lerner

Description	3. : Create a custom `AutoCloseable` resource and use try-with-resources to demonstrate that the resource closes even when an exception occurs inside the block. Also demonstrate that the original exception is reported.
Code	

```
class MyResource implements AutoCloseable {
    public MyResource() {
        System.out.println("Resource opened");
    }

    public void use() {
        System.out.println("Performing operation");
        throw new RuntimeException("Original operation failed");
    }

    @Override
    public void close() throws Exception {
        System.out.println("Closing resource");
        throw new Exception("Error while closing resource");
    }
}

public class pract8_3 {
    public static void main(String[] args) {
        try (MyResource r = new MyResource()) {
            r.use();
        } catch (Exception e) {
            System.out.println("Caught exception: " + e.getMessage());

            for (Throwable t : e.getSuppressed()) {
                System.out.println("Suppressed exception: " + t.getMessage());
            }
        }

        System.out.println("Rahil Butani 25CE011");
    }
}
```

Output

```
PS D:\JAVA_LAB-main\P8\advance> javac main.java
PS D:\JAVA_LAB-main\P8\advance> java main
Resource opened
Performing operation
Closing resource
Caught exception: Original operation failed
Suppressed exception: Error while closing resource
Dhairy Chauhan 24CE015
PS D:\JAVA_LAB-main\P8\advance>
```

p-8.3.png

Practical 9: Multithreading and Synchronization

Description	1. Multiple threads increment shared counters. The unsynchronized counter may lose updates; the synchronized counter should reach the expected total.
Code	

```
import java.util.ArrayList;
import java.util.List;

class Counter {
   int count;

   void increment() {
      count = count + 1;
   }

   void incrementSafe() {
      count = count + 1;
   }
}

public class main {
   static final int THREADS = 4;
   static final int TIMES = 100000;

   public static void main(String[] args) throws InterruptedException {
      Counter raceCounter = new Counter();
      Counter safeCounter = new Counter();

      run(raceCounter, false);
      run(safeCounter, true);

      int expected = THREADS * TIMES;
      System.out.println("Threads     : " + THREADS);
      System.out.println("Per thread  : " + TIMES);
      System.out.println("Expected    : " + expected);
      System.out.println("No sync     : " + raceCounter.count + " " + verdict(raceCounter.count
== expected));
      System.out.println("With sync   : " + safeCounter.count + " " + verdict(safeCounter.count
== expected));
   }

   static void run(Counter counter, boolean useSync) throws InterruptedException {
      List<Thread> threads = new ArrayList<>();
      for (int i = 0; i < THREADS; i++) {
         Thread t = new Thread(() -> {
            for (int j = 0; j < TIMES; j++) {
               if (useSync) {
                  counter.incrementSafe();
               } else {
                  counter.increment();
```

```
            }
          }
        });
        threads.add(t);
        t.start();
      }
      for (Thread t : threads) {
        t.join();
      }
    }

    static String verdict(boolean correct) {
      return correct ? "(correct)" : "(too low, updates were lost)";
    }
     System.out.println("Dhairy Chauhan 25CE015");
}
```

Output

```
PS D:\JAVA_LAB-main\P9\Counter race> javac main.java
main.java:58: error: <identifier> expected
     System.out.println("Dhairy Chauhan 25CE015");
                        ^
main.java:58: error: illegal start of type
     System.out.println("Dhairy Chauhan 25CE015");
                        ^
2 errors
PS D:\JAVA_LAB-main\P9\Counter race> javac main.java
PS D:\JAVA_LAB-main\P9\Counter race> java main
Threads      : 4
Per thread   : 100000
Expected     : 400000
No sync      : 119146 (too low, updates were lost)
With sync    : 285000 (too low, updates were lost)
PS D:\JAVA_LAB-main\P9\Counter race>
```

p-3.1.png

Program 9.2: Seat Booking Race Condition

Description	2. To demonstrate a race condition in a seat booking system and show how synchronization prevents more seats from being booked than are available. Ten threads try to book five seats. The unsafe booking demonstrates a race condition; synchronized booking accepts no more than five requests.
Code	class Seat { int seatsLeft = 5;

```
boolean book() throws InterruptedException {
      if (seatsLeft > 0) {
         Thread.sleep(2);
         seatsLeft = seatsLeft - 1;
         return true;
      }
      return false;
   }
}

class BookingThread extends Thread {
   Seat seat;
   boolean got;

   BookingThread(Seat seat) {
      this.seat = seat;
   }

   public void run() {
      try {
         got = seat.book();
      } catch (InterruptedException e) {
         got = false;
      }
      if (got) {
         System.out.println(getName() + " booked a seat");
      } else {
         System.out.println(getName() + " failed to book");
      }
   }
}

class SyncSeat {
   int seatsLeft = 5;

   synchronized boolean book() throws InterruptedException {
      if (seatsLeft > 0) {
         Thread.sleep(2);
         seatsLeft = seatsLeft - 1;
         return true;
      }
      return false;
   }
}

class SyncBookingThread extends Thread {
   SyncSeat seat;
```

```
    boolean got;

    SyncBookingThread(SyncSeat seat) {
        this.seat = seat;
    }

    public void run() {
        try {
            got = seat.book();
        } catch (InterruptedException e) {
            got = false;
        }
        if (got) {
            System.out.println(getName() + " booked a seat");
        } else {
            System.out.println(getName() + " failed to book");
        }
    }
}

public static void main(String[] args) throws InterruptedException {
        Seat seat = new Seat();
        BookingThread[] threads = new BookingThread[10];
        for (int i = 0; i < 10; i++) {
            threads[i] = new BookingThread(seat);
            threads[i].start();
        }
        for (int i = 0; i < 10; i++) {
            threads[i].join();
        }

        System.out.println("Seats left: " + seat.seatsLeft);

        SyncSeat syncSeat = new SyncSeat();
        SyncBookingThread[] syncThreads = new SyncBookingThread[10];
        for (int i = 0; i < 10; i++) {
            syncThreads[i] = new SyncBookingThread(syncSeat);
            syncThreads[i].start();
        }
        for (int i = 0; i < 10; i++) {
            syncThreads[i].join();
        }

        System.out.println("Seats left after synchronized booking: " +
syncSeat.seatsLeft);
         System.out.println("Dhairy Chauhan 25CE015");
    }
```

Output	

```
PS D:\JAVA_LAB-main\P9\Seat booking race> javac main.java
PS D:\JAVA_LAB-main\P9\Seat booking race> java main
Thread-2 booked a seat
Thread-4 booked a seat
Thread-0 booked a seat
Thread-1 booked a seat
Thread-5 booked a seat
Thread-3 booked a seat
Thread-6 booked a seat
Thread-7 booked a seat
Thread-8 booked a seat
Thread-9 booked a seat
Seats left: -3
Thread-12 booked a seat
Thread-19 booked a seat
Thread-14 booked a seat
Thread-15 booked a seat
Thread-17 booked a seat
Thread-13 failed to book
Thread-11 failed to book
Thread-18 failed to book
Thread-16 failed to book
Thread-10 failed to book
Seats left after synchronized booking: 0
Dhairy Chauhan 25CE015
PS D:\JAVA_LAB-main\P9\Seat booking race>
```

p-3.2.png

Program 9.3: Shared Array Sum Using Multithreading

Description	3. To split a large array among multiple threads, demonstrate incorrect results caused by a race condition, fix the issue using two approaches, and compare their execution times.

Code

```
import java.util.concurrent.atomic.AtomicLong;


public class main {
   private static final int SIZE = 10_000_000;
   private static final int THREADS = 4;
   private static final int[] ARRAY = new int[SIZE];


   static {
      for (int i = 0; i < SIZE; i++) {
         ARRAY[i] = 1;
```

```
    }
  }

  public static void main(String[] args) throws InterruptedException {
    System.out.println("Expected total: " + SIZE);
    System.out.println("-----------------------------------");

    runWrongWay();
    runFixOne();
    runFixTwo();
  }

  private static void runWrongWay() throws InterruptedException {
    WrongAdder adder = new WrongAdder();
    Thread[] threads = new Thread[THREADS];
    int chunkSize = SIZE / THREADS;

    long start = System.nanoTime();
    for (int i = 0; i < THREADS; i++) {
      int startIdx = i * chunkSize;
      int endIdx = (i == THREADS - 1) ? SIZE : startIdx + chunkSize;
      threads[i] = new Thread(() -> adder.add(startIdx, endIdx));
      threads[i].start();
    }

    for (Thread t : threads) {
      t.join();
    }
    long end = System.nanoTime();

    System.out.println("Wrong Approach Total: " + adder.total);
    System.out.println("Wrong Approach Time: " + (end - start) / 1_000_000.0 + "
ms");
    System.out.println("-----------------------------------");
  }

  private static void runFixOne() throws InterruptedException {
    SyncAdder adder = new SyncAdder();
    Thread[] threads = new Thread[THREADS];
    int chunkSize = SIZE / THREADS;
```

```
        long start = System.nanoTime();
        for (int i = 0; i < THREADS; i++) {
            int startIdx = i * chunkSize;
            int endIdx = (i == THREADS - 1) ? SIZE : startIdx + chunkSize;
            threads[i] = new Thread(() -> adder.add(startIdx, endIdx));
            threads[i].start();
        }

        for (Thread t : threads) {
            t.join();
        }
        long end = System.nanoTime();

        System.out.println("Fix 1 (Synchronized) Total: " + adder.total);
        System.out.println("Fix 1 (Synchronized) Time: " + (end - start) / 1_000_000.0 + "
ms");
        System.out.println("-----------------------------------");
    }

    private static void runFixTwo() throws InterruptedException {
        AtomicAdder adder = new AtomicAdder();
        Thread[] threads = new Thread[THREADS];
        int chunkSize = SIZE / THREADS;

        long start = System.nanoTime();
        for (int i = 0; i < THREADS; i++) {
            int startIdx = i * chunkSize;
            int endIdx = (i == THREADS - 1) ? SIZE : startIdx + chunkSize;
            threads[i] = new Thread(() -> adder.add(startIdx, endIdx));
            threads[i].start();
        }

        for (Thread t : threads) {
            t.join();
        }
        long end = System.nanoTime();

        System.out.println("Fix 2 (AtomicLong) Total: " + adder.total.get());
        System.out.println("Fix 2 (AtomicLong) Time: " + (end - start) / 1_000_000.0 + "
ms");
        System.out.println("-----------------------------------");
    }
```

```
    static class WrongAdder {
        long total = 0;

        public void add(int start, int end) {
            for (int i = start; i < end; i++) {
                total += ARRAY[i];
            }
        }
    }

    static class SyncAdder {
        long total = 0;

        public synchronized void add(int start, int end) {
            long localSum = 0;
            for (int i = start; i < end; i++) {
                localSum += ARRAY[i];
            }
            synchronized (this) {
                total += localSum;
            }
        }
    }

    static class AtomicAdder {
        AtomicLong total = new AtomicLong(0);

        public void add(int start, int end) {
            long localSum = 0;
            for (int i = start; i < end; i++) {
                localSum += ARRAY[i];
            }
            total.addAndGet(localSum);
             System.out.println("Dhairy Chauhan 25CE015");
        }
    }
}
```

Output

```
PS D:\JAVA_LAB-main\P9\advance> javac main.java
PS D:\JAVA_LAB-main\P9\advance> java main
Expected total: 10000000
---------------------------------
Wrong Approach Total: 5176139
Wrong Approach Time: 17.3447 ms
---------------------------------
Fix 1 (Synchronized) Total: 10000000
Fix 1 (Synchronized) Time: 9.3281 ms
---------------------------------
Dhairy Chauhan 25CE015
Dhairy Chauhan 25CE015
Dhairy Chauhan 25CE015
Dhairy Chauhan 25CE015
Fix 2 (AtomicLong) Total: 10000000
Fix 2 (AtomicLong) Time: 11.1764 ms
---------------------------------
PS D:\JAVA_LAB-main\P9\advance>
```

p-3.3.png

Practical 10: Thread Pools and Inter-Thread Communication

Description	1. Submit ten tasks to a fixed pool of three threads. The executor reuses its small set of worker threads and waits for all tasks to finish.

Code	iimport java.util.concurrent.*;
	
	public class main {
	  public static void main(String[] args) throws InterruptedException {
	  ExecutorService pool = Executors.newFixedThreadPool(3);
	
	  for (int i = 1; i <= 10; i++) {
	  final int id = i;
	  pool.execute(() -> {
	  System.out.println("Task " + id + " running on " +
	  Thread.currentThread().getName());
	  try {
	  Thread.sleep(200);
	  } catch (InterruptedException e) {
	  Thread.currentThread().interrupt();
	  }
	  });
	  }
	
	  pool.shutdown();
	  if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
	  pool.shutdownNow();
	  }

```
        System.out.println("All tasks completed");
        System.out.println("Dhairy Chauhan 25CE015");

    }

}
```

Output

```
PS D:\JAVA_LAB-main\P10\Thread-pool runner> javac main.java
PS D:\JAVA_LAB-main\P10\Thread-pool runner> java main
Task 3 running on pool-1-thread-3
Task 2 running on pool-1-thread-2
Task 1 running on pool-1-thread-1
Task 4 running on pool-1-thread-3
Task 5 running on pool-1-thread-1
Task 6 running on pool-1-thread-2
Task 7 running on pool-1-thread-2
Task 8 running on pool-1-thread-1
Task 9 running on pool-1-thread-3
Task 10 running on pool-1-thread-2
All tasks completed
Dhairy Chauhan 25CE015
PS D:\JAVA_LAB-main\P10\Thread-pool runner>
```

p-10.1.png

Program 10.2: Producer–Consumer

Description	2. Producer–consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost.

Code

```
iimport java.util.LinkedList;

import java.util.Queue;


class Buffer {

   private final Queue<Integer> queue = new LinkedList<>();

   private final int capacity = 2;


   synchronized void put(int value) throws InterruptedException {

      while (queue.size() == capacity) {

         wait();

      }

      queue.add(value);

      System.out.println("Produced: " + value);

      notifyAll();

   }


   synchronized int take() throws InterruptedException {

      while (queue.isEmpty()) {

         wait();

      }

      int value = queue.remove();

      System.out.println("Consumed: " + value);

      notifyAll();

      return value;

   }

}
```

```
public class main{
    public static void main(String[] args) throws InterruptedException {
        Buffer buffer = new Buffer();

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.put(i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.take();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        producer.start();
        consumer.start();
        producer.join();
```

```
        consumer.join();

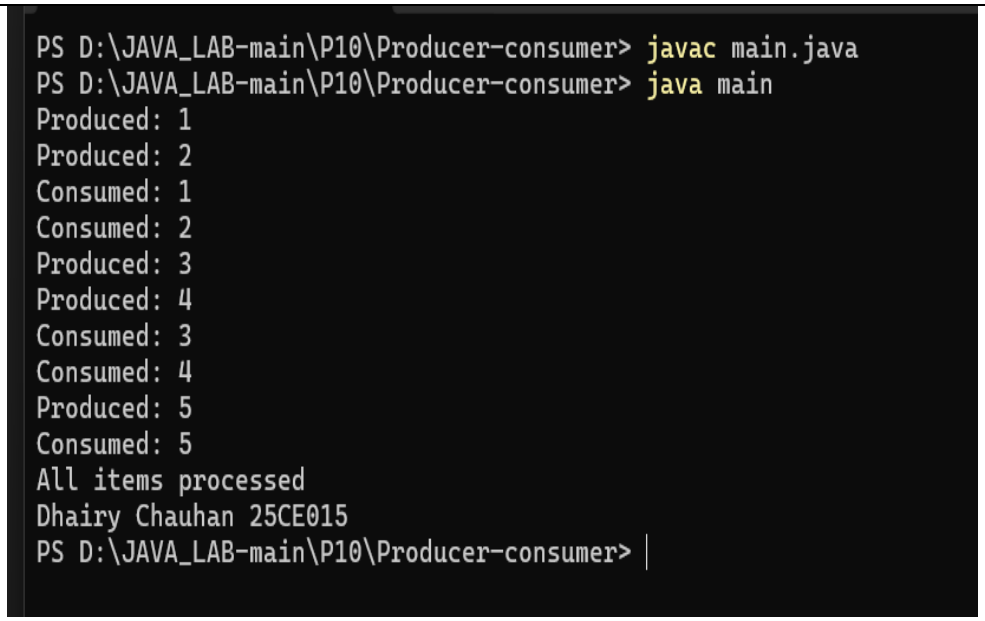


        System.out.println("All items processed");

        System.out.println("Dhairy Chauhan 25CE015");

    }

}
```

Output	 *P-10.2.png*

Program 10.3: Deadlock and Prevention Using Lock Ordering

Description	3. To demonstrate deadlock when two threads lock shared bank accounts in opposite orders, then prevent deadlock by always locking the account with the smaller account number first.

Code	import java.util.concurrent.CountDownLatch;
	
	class Account {
	  int accountNumber;
	  int balance;
	
	  Account(int accountNumber, int balance) {
	  this.accountNumber = accountNumber;
	  this.balance = balance;
	  }
	}
	
	public class main{
	  static void unsafeTransfer(Account from, Account to,
	  CountDownLatch ready) {
	  synchronized (from) {
	  ready.countDown();
	
	  try {
	  ready.await();
	  } catch (InterruptedException e) {
	  Thread.currentThread().interrupt();
	  return;

```
        }

        synchronized (to) {
            from.balance -= 100;
            to.balance += 100;
        }
    }
}

static void safeTransfer(Account from, Account to) {
    Account first = from.accountNumber < to.accountNumber
            ? from : to;
    Account second = from.accountNumber < to.accountNumber
            ? to : from;

    synchronized (first) {
        synchronized (second) {
            from.balance -= 100;
            to.balance += 100;
        }
    }
}

public static void main(String[] args) throws InterruptedException {
    Account a = new Account(1, 1000);
    Account b = new Account(2, 1000);
    CountDownLatch ready = new CountDownLatch(2);

    Thread t1 = new Thread(() -> unsafeTransfer(a, b, ready));
    Thread t2 = new Thread(() -> unsafeTransfer(b, a, ready));

    t1.setDaemon(true);
    t2.setDaemon(true);

    t1.start();
    t2.start();

    t1.join(500);
    t2.join(500);

    if (t1.isAlive() || t2.isAlive()) {
        System.out.println("Deadlock detected in unsafe transfer");
    } else {
        System.out.println("Unsafe transfers completed");
    }

    Account c = new Account(1, 1000);
```

```
        Account d = new Account(2, 1000);

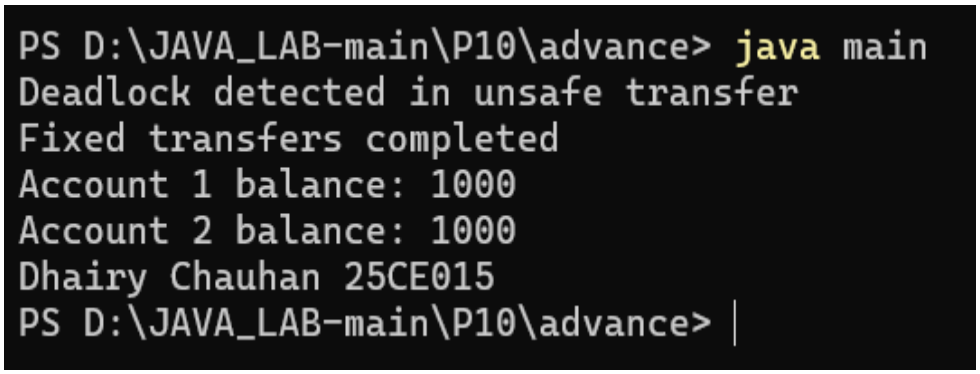
        Thread t3 = new Thread(() -> safeTransfer(c, d));
        Thread t4 = new Thread(() -> safeTransfer(d, c));

        t3.start();
        t4.start();

        t3.join();
        t4.join();

        System.out.println("Fixed transfers completed");
        System.out.println("Account 1 balance: " + c.balance);
        System.out.println("Account 2 balance: " + d.balance);
        System.out.println("Dhairy Chauhan 25CE015");
    }
}
```

Output



P-4.3.png

Practical 11: File Handling

Description	1. To implement `Serializable`, save an array of objects using `ObjectOutputStream`, load it using `ObjectInputStream`, and show that a `transient` field is not saved.

Code

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;

class Student implements Serializable {
    private static final long serialVersionUID = 1L;

    String name;
    int marks;
    transient String password;

    Student(String name, int marks, String password) {
        this.name = name;
        this.marks = marks;
        this.password = password;
    }

    public String toString() {
        return name + " marks=" + marks + " password=" + password;
    }
}

public class Save {
    public static void main(String[] args) throws IOException, ClassNotFoundEx-
ception {
        Student[] students = {
            new Student("Aarav", 88, "s3cret-a"),
            new Student("Bhavna", 72, "s3cret-b"),
            new Student("Chirag", 95, "s3cret-c")
        };

        System.out.println("Before saving:");
        for (Student s : students) {
            System.out.println("  " + s);
        }

        try (ObjectOutputStream out = new ObjectOutputStream(new FileOut-
putStream("students.dat"))) {
            out.writeObject(students);
        }

        Student[] loaded;
        try (ObjectInputStream in = new ObjectInputStream(new FileIn-
putStream("students.dat"))) {
            loaded = (Student[]) in.readObject();
        }

        System.out.println("\nAfter loading:");
        boolean allMatch = true;
```

```
        for (int i = 0; i < loaded.length; i++) {
            System.out.println("  " + loaded[i]);
            if (!loaded[i].name.equals(students[i].name) || loaded[i].marks != 
students[i].marks) {
                allMatch = false;
            }
        }


        System.out.println("\nPersistent fields (name, marks) survived: " + all-
Match);
        System.out.println("transient password survived: " + (loaded[0].password 
!= null));
        System.out.println("(password is transient, so it is not saved and comes 
back null)");
        System.out.println("Dhairy chauhan 25CE015");
    }
}
```

Output

```
PS D:\JAVA_LAB-main\P11\Save> javac Save.java
PS D:\JAVA_LAB-main\P11\Save> java Save
Before saving:
  Aarav marks=88 password=s3cret-a
  Bhavna marks=72 password=s3cret-b
  Chirag marks=95 password=s3cret-c

After loading:
  Aarav marks=88 password=null
  Bhavna marks=72 password=null
  Chirag marks=95 password=null

Persistent fields (name, marks) survived: true
transient password survived: false
(password is transient, so it is not saved and comes back null)
Dhairy chauhan 25CE015
PS D:\JAVA_LAB-main\P11\Save>
```

P-5.1.png

Program 11.2: Log Analyzer Using NIO

Description

2. To read multiple text files line by line using `Path` and `Files`, count total lines and lines containing a keyword, and display each file's size.

Code

```
import java.nio.file.*;
import java.io.IOException;
import java.util.*;


public class main{
    public static void main(String[] args) throws IOException {
        Path folder = Paths.get("logs");
        Files.createDirectories(folder);


        Files.write(folder.resolve("app1.txt"), Arrays.asList(
            "User login successful",
            "Error in database",
            "User logout"
        ));
```

```
        Files.write(folder.resolve("app2.txt"), Arrays.asList(
            "Error in payment",
            "User login successful",
            "System started"
        ));

        String keyword = "error";
        int totalLines = 0;
        int matchedLines = 0;

        try (DirectoryStream<Path> files =
                Files.newDirectoryStream(folder, "*.txt")) {
            for (Path file : files) {
                List<String> lines = Files.readAllLines(file);
                int size = lines.size();
                int matches = 0;

                for (String line : lines) {
                    totalLines++;
                    if (line.toLowerCase().contains(keyword)) {
                        matchedLines++;
                        matches++;
                    }
                }

                System.out.println("File: " + file.getFileName());
                System.out.println("Size: " + Files.size(file) + " bytes");
                System.out.println("Lines: " + size);
                System.out.println("Keyword matches: " + matches);
            }
        }

        System.out.println("Total lines: " + totalLines);
        System.out.println("Lines containing error: " + matchedLines);
        System.out.println("Dhairy Chauhan 25ce015");
    }
}
```

Output

```
PS D:\JAVA_LAB-main\P11\Log analyzer> javac main.java
PS D:\JAVA_LAB-main\P11\Log analyzer> java main
File: app1.txt
Size: 55 bytes
Lines: 3
Keyword matches: 1
File: app2.txt
Size: 57 bytes
Lines: 3
Keyword matches: 1
Total lines: 6
Lines containing error: 2
Dhairy Chauhan 25ce015
PS D:\JAVA_LAB-main\P11\Log analyzer>
```

p-5.2.png

Program 11.3: Folder Report Using NIO

Description	3. To walk through a folder tree, collect each file's name, size and modification time, and write the details into a report file.

Code

```
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.io.IOException;
import java.util.*;

public class main{
    public static void main(String[] args) throws IOException {
        Path folder = Paths.get("logs");
        Files.createDirectories(folder);
        Path report = Paths.get("folder_report.txt");

        List<String> details = new ArrayList<>();
        details.add("Folder Report");

        Files.walkFileTree(folder, new SimpleFileVisitor<Path>() {
            @Override
            public FileVisitResult visitFile(Path file,
                    BasicFileAttributes attrs) {
                try {
                    details.add("Name: " + file);
                    details.add("Size: " + attrs.size() + " bytes");
                    details.add("Modified: " + attrs.lastModifiedTime());
                } catch (Exception e) {
                    details.add("Could not read: " + file);
                }
                return FileVisitResult.CONTINUE;
            }
        });

        Files.write(report, details);
```

```
        System.out.println("Folder scan completed");
        System.out.println("Report saved to: " + report.toAbsolutePath());
        System.out.println("Files listed: " + (details.size() - 1) / 4);
        System.out.println("Dhairy Chauhan 25CE015");
    }
}
```

Output

```
PS D:\JAVA_LAB-main\P11\advance> javac main.java
PS D:\JAVA_LAB-main\P11\advance> java main
Folder scan completed
Report saved to: D:\JAVA_LAB-main\P11\advance\folder_report.txt
Files listed: 0
Dhairy Chauhan 25CE015
PS D:\JAVA_LAB-main\P11\advance>
```

P-11.3.png

Practical 12: Collections Framework

Description	1. To count the frequency of each word using a `HashMap` and sort the words by frequency in descending order using a `Comparator`

Code

```
import java.util.*;

public class main{
    public static void main(String[] args) {
        String text = "java is easy and java is useful and java is popular";
        String[] words = text.toLowerCase().split("\\W+");

        HashMap<String, Integer> freq = new HashMap<>();

        for (String word : words) {
            freq.put(word, freq.getOrDefault(word, 0) + 1);
        }

        List<Map.Entry<String, Integer>> list =
                new ArrayList<>(freq.entrySet());

        list.sort((a, b) -> {
            int result = b.getValue().compareTo(a.getValue());
            if (result == 0) {
                return a.getKey().compareTo(b.getKey());
            }
            return result;
        });

        for (Map.Entry<String, Integer> entry : list) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }

        System.out.println("Dhairy Chauhan 25CE015");
    }
}
```

Output

```
PS D:\JAVA_LAB-main\P12\Word frequency> javac main.java
PS D:\JAVA_LAB-main\P12\Word frequency> java main
is : 3
java : 3
and : 2
easy : 1
popular : 1
useful : 1
Dhairy Chauhan 25CE015
PS D:\JAVA_LAB-main\P12\Word frequency>
```

p-6.1.png

Program 12.2: Notification System

Description	2. Unique visitors: from a list of login names with repeats, use a HashSet to count unique users and a LinkedHashSet to keep first-seen order; print the unique count and the first three distinct names.
Code	

```
import java.util.*;

public class main{
    public static void main(String[] args) {
        String[] visitors = {
            "Dhairy", "aman", "priya", "rahil",
            "jay", "aman", "neha", "priya"
        };

        HashSet<String> unique = new HashSet<>();
        LinkedHashSet<String> ordered = new LinkedHashSet<>();

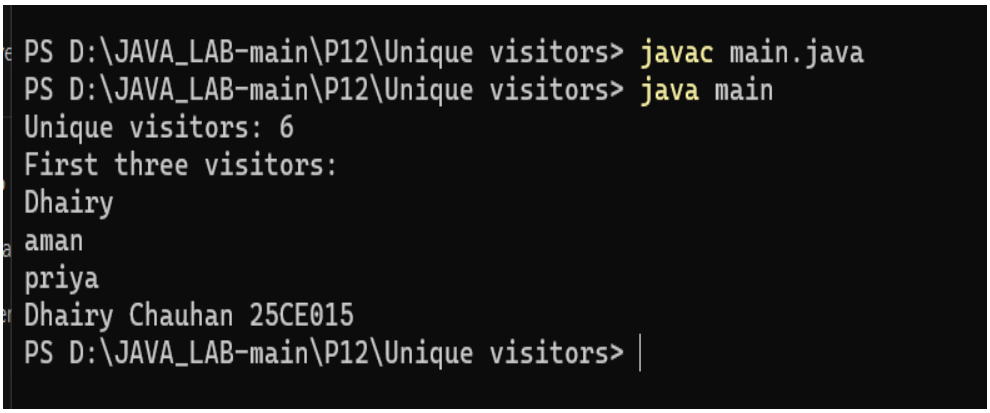
        for (String name : visitors) {
            unique.add(name);
            ordered.add(name);
        }

        System.out.println("Unique visitors: " + unique.size());
        System.out.println("First three visitors:");

        int count = 0;
        for (String name : ordered) {
            if (count == 3) {
                break;
            }
            System.out.println(name);
            count++;
        }

        System.out.println("Dhairy Chauhan 25CE015");
    }
}
```

Output



p-12.2.png

Program 12.3: Player Leaderboard

Description	3. Build a leaderboard from player scores held in a Map and print the top-N sorted by score (ties broken by name); choose the data and sorting details.
Code	

```
import java.util.*;

public class main{
    public static void main(String[] args) {
        Map<String, Integer> scores = new HashMap<>();

        scores.put("Dhairy", 95);
        scores.put("Aman", 88);
        scores.put("Priya", 95);
        scores.put("Jay", 76);
        scores.put("Neha", 88);

        int n = 3;

        List<Map.Entry<String, Integer>> players =
                new ArrayList<>(scores.entrySet());

        players.sort((a, b) -> {
            int result = b.getValue().compareTo(a.getValue());
            if (result == 0) {
                return a.getKey().compareTo(b.getKey());
            }
            return result;
        });

        System.out.println("Top " + n + " players:");

        for (int i = 0; i < Math.min(n, players.size()); i++) {
            Map.Entry<String, Integer> p = players.get(i);
            System.out.println(p.getKey() + " - " + p.getValue());
        }

        System.out.println("Dhairy Chauhan 25ce015");
    }
}
```

Output

```
PS D:\JAVA_LAB-main\P12\advance> javac main.java
PS D:\JAVA_LAB-main\P12\advance> java main
Top 3 players:
Dhairy - 95
Priya - 95
Aman - 88
Dhairy Chauhan 25ce015
PS D:\JAVA_LAB-main\P12\advance>
```

p-12.3.png

Practical 13: Generics

Description	1. To create a generic `Box<T>` and `Pair<A,B>`, demonstrate type safety with different data types, and write a generic method that returns the first element of an array.
Code	

```
class Box<T> {
    private T value;

    void set(T value) {
        this.value = value;
    }

    T get() {
        return value;
    }
}

class Pair<A, B> {
    A first;
    B second;

    Pair(A first, B second) {
        this.first = first;
        this.second = second;
    }

    void display() {
        System.out.println(first + " - " + second);
    }
}

public class main{
```

```
    static <T> T first(T[] arr) {
        if (arr == null || arr.length == 0) {
            return null;
        }
        return arr[0];
    }

    public static void main(String[] args) {
        Box<String> name = new Box<>();
        name.set("Rahil");

        Box<Integer> marks = new Box<>();
        marks.set(95);

        System.out.println("Name: " + name.get());
        System.out.println("Marks: " + marks.get());

        Pair<String, Integer> student =
                new Pair<>("Rahil", 25);
        student.display();

        String[] names = {"Aman", "Rahil", "Priya"};
        Integer[] numbers = {10, 20, 30};

        System.out.println("First name: " + first(names));
        System.out.println("First number: " + first(numbers));
        System.out.println("Dhairy Chauhan 25CE015");
    }
}
```

Output

```
PS D:\JAVA_LAB-main\P13\Generic containers> javac main.java
PS D:\JAVA_LAB-main\P13\Generic containers> java main
Name: Rahil
Marks: 95
Rahil - 25
First name: Aman
First number: 10
Dhairy Chauhan 25CE015
PS D:\JAVA_LAB-main\P13\Generic containers>
```

p-6.1.png

Program 13.2: Bounded Generic Average

Description	2. To write a generic method using the bound `N extends Number` that calculates averages for `Integer[]`, `Double[]` and `Long[]`. Non-numeric types are not accepted by this method.

Code

```
public class main{
    static <N extends Number> double average(N[] values) {
        if (values == null || values.length == 0) {
            return 0;
        }
```

```
        double sum = 0;

        for (N value : values) {
            sum += value.doubleValue();
        }

        return sum / values.length;
    }

    public static void main(String[] args) {
        Integer[] a = {10, 20, 30, 40};
        Double[] b = {2.5, 3.5, 4.5};
        Long[] c = {100L, 200L, 300L};

        System.out.println("Integer average: " + average(a));
        System.out.println("Double average: " + average(b));
        System.out.println("Long average: " + average(c));
        System.out.println("Dhairy Chauhan 25CE015");
    }
}
```

Output

```
PS D:\JAVA_LAB-main\P13\Bounded stats> javac main.java
PS D:\JAVA_LAB-main\P13\Bounded stats> java main
Integer average: 25.0
Double average: 3.5
Long average: 200.0
Dhairy Chauhan 25CE015
PS D:\JAVA_LAB-main\P13\Bounded stats>
```

p-12.2.png

Program 13.3: Generic Stack Using an Array

Description

3. To implement a generic stack backed by an array, reuse it with different data types, and handle attempts to remove an item from an empty stack.

Code

```
class MyStack<T> {
    private Object[] items;
    private int top = -1;

    MyStack(int size) {
        items = new Object[size];
    }

    void push(T value) {
        if (top == items.length - 1) {
            System.out.println("Stack is full");
            return;
        }
        items[++top] = value;
```

```
    }

    @SuppressWarnings("unchecked")
    T pop() {
        if (top == -1) {
            System.out.println("Stack is empty");
            return null;
        }
        T value = (T) items[top];
        items[top--] = null;
        return value;
    }

    boolean isEmpty() {
        return top == -1;
    }
}

public class main{
    public static void main(String[] args) {
        MyStack<Integer> numbers = new MyStack<>(5);
        numbers.push(10);
        numbers.push(20);
        numbers.push(30);

        System.out.println("Number popped: " + numbers.pop());
        System.out.println("Number popped: " + numbers.pop());

        MyStack<String> names = new MyStack<>(3);
        names.push("Rahil");
        names.push("Aman");

        System.out.println("Name popped: " + names.pop());
        System.out.println("Name popped: " + names.pop());
        System.out.println("Name popped: " + names.pop());

        System.out.println("Stack empty: " + names.isEmpty());
        System.out.println("Dhairy Chauhan 25CE015");
    }
}
```

Output

```
PS D:\JAVA_LAB-main\P13\advance> javac main.java
PS D:\JAVA_LAB-main\P13\advance> java main
Number popped: 30
Number popped: 20
Name popped: Aman
Name popped: Rahil
Stack is empty
Name popped: null
Stack empty: true
Dhairy Chauhan 25CE015
PS D:\JAVA_LAB-main\P13\advance>
```

	p-13.3.png